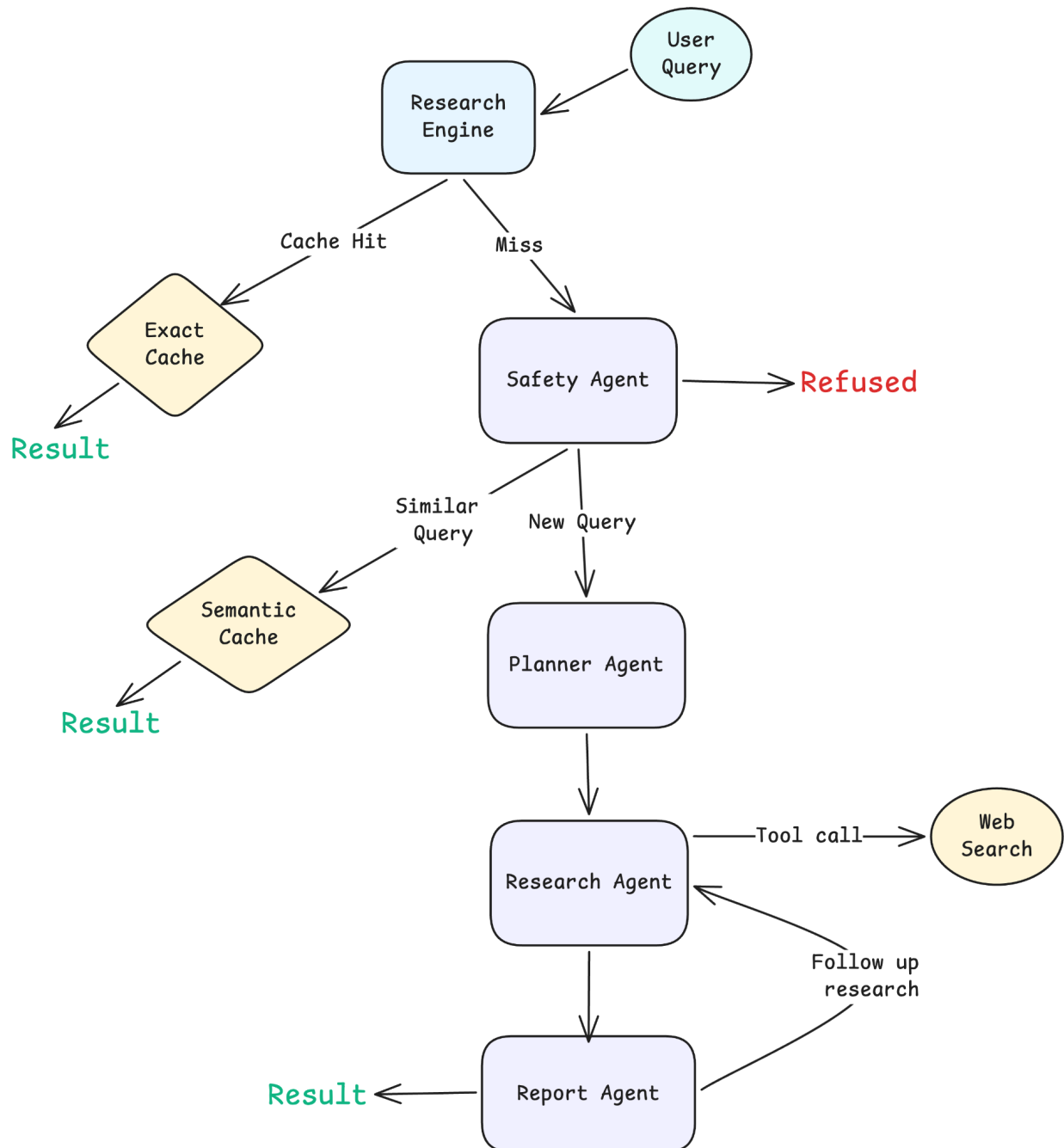


Multi agent orchestration for research and report

A multi-agent AI research orchestration system that collaborates across specialized agents to safely generate structured research reports from user queries.

Architecture



I have 4 agents working on user goal:

1. **Safety Agent:** Lightweight agent which validates user query, detects any prompt injections, removes suspicious instructions and creates cleaned up query. It decides whether to allow, refuse or proceed with caution.
2. **Planner Agent:** It takes the cleaned query from safety agent and re-writes the goal clearly. Breaks down the goal into 3-5 research tasks and decides which information is needed.
3. **Research Agent:** Goes through the tasks and performs web search, summarizes source findings and stores URLs/titles. It does not make the final conclusion, just research about the task.
4. **Report Agent:** It reviews the research and checks if the evidence is enough, identifies gaps or uncertainty and optionally requests follow up research. Then creates final report with citations and limitations for the user.

Communication Pattern

Engine file which is the main orchestrator contains shared memory state which is updated sequentially based each agent's output.

Why only short term shared memory? its one shot query, keeping stateless, not multiple users, no need for keeping conversation. also it would require me to have privacy handling if I store anything which is sensitive

Security, Safety and Guardrails

- User query is first normalized and then passed into Safety agent which verifies if there are any risk flags.
- I have created XML prompts which are built and injected to each agent at runtime. Using XML gives good structure to properly define roles, constrains and tasks.
- I used zod schemas for strict structured object output generated by LLM and this is pretty nicely enforced by Vercel's AI SDK. The structure is also explained in the prompt.
- If the agent's output is even slightly different from our intended schema then zod will fail immediately.
- **Logging:** I created custom logging class which is injected from engine to agent and cache classes. There are different levels like info, error, metric. So the engine will log the pipeline flow, agents logs the LLM metrics and retry failures.
- **PII:** All the api keys and other configurable params are loaded through environment variables.

- **Tool limit:** Vercel provides stopWhen() for each agent which I have configured with hard limit so agent can only do limited web search tool calls rather than burning my token balance.

Implementation Approach

- **Language:** I chose TypeScript because its kind of becoming the most supported for ai systems and also have type safety for better contracts. no hate for python, just wanted to learn typescript
- **AI SDK:** I used Vercel AI SDK because blends with typescript well, have their cloud hosting, gives access to all the models, excellent documentation.
- **Agent Orchestration:** I created custom orchestration for the pipeline to learn the flow and have flexibility. I could have used LangGraph or some other framework which would have abstracted away all my learning.
- Agents are instantiated through Vercel's OpenAI provider when the engine starts. They coordinate through shared memory state and are single shot call based on the goal provided.
- **Retry/Failure:** I created custom withRetry(agent, retries) flow which takes the agent and also number of retries. Safety agent doesn't get any retry because if safety fails then the we shouldn't proceed at all. If planner/research/report agent fails which could be due to server issue or output schema didn't match then we retry with same memory state. Number of retries are configurable. Engine tracks the which agent is running and updates the state if any agent fails with specific logs.
- **Testing:** I have created dummy pass through data which was an actual output of the whole pipeline. We can turn it on through environment variable and also control each specific agent using it. Any changes to codeflow must pass all the pipeline with dummy data.

Use of AI/LLMs and Collaboration

- Safety agent verifies and decides if the user query to be passed into the pipeline or not. Then planner agent will break down the goal and create research tasks. Research agent will use web search tool and gather info as guided in the prompt. Report agent will review and generate final report.
- **Follow up research agent:** I noticed that sometimes research agent doesn't do well job in gathering information so report agent decides a follow up research with additional tasks. Then research agent will complete those tasks as well and merge into previous findings.
- **Autonomy and Control:** Whole flow is completely controlled to make the pipeline as deterministic as possible. Policies and roles are very strictly defined in the prompt and

zod schema makes sure that the output is structured. Tool calls have hard limit through parameters provided by the SDK. Follow up research is only one time allowed.

Caching

- **Exact Cache:** Engine output is cached with user's normalized query as key in Redis with 24 Hrs TTL. It prevents user from spamming run button. Engine always checks if the query is present in the exact cache before proceeding. Cache is stored only when the pipeline result is success and never on error.
- **Semantic Cache:** Sometimes same or different will have the same goal but differently worded so I have created custom semantic cache orchestration based on cosine similarity. I tried with 0.9 threshold but that was too strict for short queries so 0.82 is the sweet spot but again this is configurable. I used Redis server for it instead of vector because then I wouldn't have learned custom orchestration and also it's easier to maintain one database.
- Semantic cache is only checked after Safety Agent allows the query to pass down otherwise we won't store it.
- Once it hits we store that query in the exact cache as well.
- TTL is kept 24 hrs because that's reasonable enough for the web search data to be stored.

Frustrating issue I faced while integrating Redis: I have index.ts with api endpoint exposed using Hono, now about this redis

I have created a file redis.ts inside cache/ which creates the redis client

now the problem is that I don't wanna create redis globally because then my app will crash because of it and I have to handle that and create extra failure handling for it.

Other solution I tried was to do lazyConnect so that when first set is called then it will connect to redis but that caused race conditions. Which failed first hit but then Redis warmed up for the next one, still not good.

So to solve this I am initializing the Redis along with my Hono api server and allowing the Redis to fail or pass but the agents will still work just they will be little more expensive. Also the logs inside catch for redis are intentionally empty otherwise they will fill up my cloud storage.

Github: <https://github.com/Lakshyasaharan5/multi-agent-research-engine>

AWS url: <http://3.145.89.223:3000/>

Health Check:

GET /

Run Research Pipeline:

POST /api/research

Example Body: { "query": "Research the impact of artificial intelligence on software engineering jobs, including current trends, risks, opportunities and future outlook." }